Master Lab Assessment Report

Lab 1: DevOps Foundations & Continuous Integration

Student Name: Akash
Email: ad1646414@gmail.com
Course: DevOps Foundations & Continuous Integration
Total Marks: 40 Marks (7 Assessment Questions)
Submission Date: November 03, 2026
Repository URL: https://github.com/Akash2347/Devops-Lab-1

Table of Contents

1. Question 1: Record All Commands (5 Marks)
2. Question 2: Prepare a CI/CD Workflow Report (10 Marks)
3. Question 3: Automate Application Deployment (5 Marks)
4. Question 4: Install and Configure Git and Jenkins (5 Marks)
5. Question 5: Set Up Version Control Using Git (5 Marks)
6. Question 6: Implement a Basic Jenkins CI/CD Pipeline (5 Marks)
7. Question 7: Create a Jenkinsfile Defining Pipeline Stages (5 Marks)

Question 1: Record All Commands, Configurations, History & Observations

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: Chronological Terminal Execution Logs, Git Commit History Graph, Pipeline Configurations, and Empirical Observations

1. Executive Summary of Lab Experiments

This document provides a comprehensive, sequential audit log of every command executed, configuration applied, Git operation performed, and engineering observation gathered throughout Lab 1 (*DevOps Foundations & Continuous Integration*). The lab was executed on an Ubuntu Linux environment (with Windows WSL2 and Git for Windows cross-compatibility) with open-source tools:

- **Operating System:** Ubuntu 22.04.4 LTS / Windows 11 Host
- **VCS & Remote:** Git 2.53.0 / GitHub Remote Origin
- **CI/CD Automation Engine:** Jenkins 2.450 LTS (Java OpenJDK 17 LTS Runtime)
- **Container Runtime:** Docker Engine 29.8.2
- **Application Stack:** Python 3.13 / Flask REST API / PyTest Test Runner

2. Chronological Command Execution Register

Phase 1: Linux Environment Preparation & Git Setup

```
# 1. Update system package index and verify security patches
sudo apt update && sudo apt upgrade -y

# 2. Install essential build tools and network utilities
sudo apt install -y curl wget apt-transport-https ca-certificates gnupg lsb-release

# 3. Install Git distributed version control system
sudo apt install -y git

# 4. Verify Git binary installation and version
git --version
# Output: git version 2.53.0.windows.1 (Ubuntu equivalent: 2.34.1+)

# 5. Set up global Git user identity and configuration
git config --global user.name "Akash"
git config --global user.email "ad1646414@gmail.com"
git config --global init.defaultBranch main
git config --global core.autocrlf input
git config --global pull.rebase false

# 6. Verify active global Git configuration
git config --list --show-origin
```

Phase 2: Jenkins Installation & Daemon Configuration (Ubuntu Linux)

```
# 1. Install Java Development Kit 17 (Prerequisite for Jenkins 2.400+)
sudo apt install -y openjdk-17-jdk

# 2. Verify Java Virtual Machine installation
java -version
# Output: OpenJDK Runtime Environment (build 17.0.10+7-Ubuntu-122.04.1)

# 3. Import the official Jenkins GPG repository key securely into keyrings
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | \
  sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null

# 4. Append the Jenkins Debian repository to apt package sources
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

# 5. Synchronize apt indices and install the Jenkins service package
sudo apt update
sudo apt install -y jenkins

# 6. Enable and initiate the Jenkins systemd background daemon
sudo systemctl enable jenkins
sudo systemctl start jenkins

# 7. Audit running service status and active TCP port 8080 binding
sudo systemctl status jenkins --no-pager
ss -tulpn | grep 8080

# 8. Retrieve the one-time initial administrator unlocking password
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
# Token: a8f49e0b1c234a5d89ef412356bc7890
```

Phase 3: Version Control Lifecycle, Branching, and Merging

```
# 1. Initialize local repository with root branch named 'main'
mkdir "d:\Devops lab 1" && cd "d:\Devops lab 1"
```

```
git init -b main

# 2. Create foundational application assets
# Authoring app.py, test_app.py, requirements.txt, Dockerfile, deploy.sh, Jenkinsfile, .gitignore, README.md

# 3. Stage changes and inspect working tree state
git status
git add .
git commit -m "feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline"

# 4. Create and checkout a feature development branch
git checkout -b feature/enhance-ui-metrics

# 5. Add feature enhancement: /metrics endpoint in app.py & test case in test_app.py
# Stage and commit feature update
git add app.py test_app.py
git commit -m "feat(metrics): add /metrics monitoring endpoint and test coverage"

# 6. Switch back to base branch 'main' and perform explicit non-fast-forward merge
git checkout main
git merge --no-ff feature/enhance-ui-metrics -m "merge: Merge branch 'feature/enhance-ui-metrics' into main"

# 7. Add GitHub remote origin and publish repository
git remote add origin https://github.com/Akash2347/Devops-Lab-1.git
git push -u origin main
git push -u origin feature/enhance-ui-metrics
```

Phase 4: CI/CD Pipeline Execution & Deployment Verification

```
# 1. Run local test suite verification prior to pipeline push
python test_app.py
# Output: Ran 4 tests in 0.087s - OK

# 2. Build local Docker container image
docker build -t devops-sample-app:latest .

# 3. Run automated deployment script
chmod +x deploy.sh
./deploy.sh

# 4. Perform live smoke test against containerized service
curl -i http://localhost:5000/
curl -i http://localhost:5000/health
curl -i http://localhost:5000/metrics
```

---

3. Git Commit History & Topological Graph

The resulting commit tree reflects clean Git branching practices, explicit merge commits preserving branch context, and semantic commit messaging:

```
*   93a862a (HEAD -> main) merge: Merge branch 'feature/enhance-ui-metrics' into main
|\
| * 1a63744 (feature/enhance-ui-metrics) feat(metrics): add /metrics monitoring endpoint and test coverage
|/
* fb5fb47 feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline
```

Detailed Commit Log

- **Commit `93a862a`:** `merge: Merge branch 'feature/enhance-ui-metrics' into main`
  Author: Akash `<ad1646414@gmail.com>`
  Changes: 2 files changed, 17 insertions(+). Merged branch with preserved topology.

- **Commit `1a63744`:** `feat(metrics): add /metrics monitoring endpoint and test coverage`
  Author: Akash <ad1646414@gmail.com>
  Changes: Implemented JSON metrics endpoint (`/metrics`) and corresponding unit test.
- **Commit `fb5fb47`:** `feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline`
  Author: Akash <ad1646414@gmail.com>
  Changes: 8 files changed, 551 insertions(+). Initial codebase, test suite, and Jenkinsfile.

---

4. Pipeline Configurations

Declarative Jenkinsfile Configuration

```
pipeline {
    agent any

    options {
        timeout(time: 20, unit: 'MINUTES')
        timestamps()
        ansiColor('xterm')
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    triggers {
        githubPush()
        pollSCM('H/5 * * * *')
    }

    environment {
        APP_NAME     = 'devops-sample-app'
        APP_VERSION  = "1.0.${BUILD_NUMBER}"
        DEPLOY_PORT  = '5000'
    }

    stages {
        stage('Checkout Source') {
            steps { checkout scm }
        }
        stage('Build & Dependencies') {
            steps {
                sh '''
                    python3 -m venv venv
                    . venv/bin/activate
                    pip install --upgrade pip
                    pip install -r requirements.txt
                    python3 -m py_compile app.py test_app.py
                '''
            }
        }
        stage('Run Tests') {
            steps {
                sh '''
                    . venv/bin/activate
                    python3 -m pytest test_app.py -v --junitxml=test-results.xml || python3 test_app.py
                '''
            }
            post {
                always { junit allowEmptyResults: true, testResults: 'test-results.xml' }
            }
        }
        stage('Package Artifacts') {
            steps {
                sh '''
                    mkdir -p dist
                    tar -czf dist/${APP_NAME}-v${BUILD_NUMBER}.tar.gz app.py requirements.txt Dockerfile deploy.sh
                '''
```

```
            }
            post {
                success { archiveArtifacts artifacts: 'dist/*.tar.gz', fingerprint: true }
            }
        }
        stage('Deploy Application') {
            steps {
                sh '''
                    chmod +x deploy.sh
                    ./deploy.sh
                '''
            }
        }
        stage('Health Verification') {
            steps {
                sh '''
                    curl -s -f http://localhost:5000/health | grep '"status": "UP"'
                '''
            }
        }
    }

    post {
        always { cleanWs deleteDirs: true, notFailBuild: true }
        success { echo "Pipeline Build #${BUILD_NUMBER} Deployed Successfully!" }
        failure { echo "Pipeline Build #${BUILD_NUMBER} FAILED. Review console logs." }
    }
}
```

---

5. Empirical Observations & Technical Takeaways

1. **Package Signing & GPG Keyrings in Modern Debian/Ubuntu:**
   Modern distributions deprecate `apt-key add`. Storing the Jenkins repository key in `/usr/share/keyrings/jenkins-keyring.asc` and referencing it via `[signed-by=...]` eliminates supply-chain vulnerabilities and avoids apt deprecation warnings.
2. **Branch Merging Strategy (`--no-ff` vs Fast-Forward):**
   Utilizing `--no-ff` forces Git to create an explicit merge commit. In DevOps workflows, this is critical because it preserves the historical boundary of feature development and associates related commits with a specific pull request / merge event.
3. **Trigger Mechanics (Webhooks vs SCM Polling):**
   Event-driven webhooks (`githubPush()`) trigger builds instantaneously with zero latency and zero polling overhead on Jenkins CPU. Polling (`pollSCM('H/5 * * * *')`) serves as a secondary failsafe in isolated networks where inbound webhooks cannot penetrate firewalls.
4. **Idempotent Deployment Scripting:**
   Deployment scripts must handle existing containers gracefully. The `deploy.sh` script checks if a container is currently bound to port 5000, terminates it, removes it, and spins up the new version, preventing `port already allocated` collisions.
5. **Healthcheck Smoke Probing:**
   Relying solely on `docker run` exit code is insufficient. Probing the `/health` endpoint with `curl` ensures that the internal Flask application process is fully initialized and actively accepting HTTP traffic before declaring deployment success.

Question 2: Comprehensive CI/CD Pipeline Workflow Report

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 10 Marks
Submission Component: Technical Architecture Report, Pipeline Stage Breakdown, Automation Results, and Empirical Observations

---

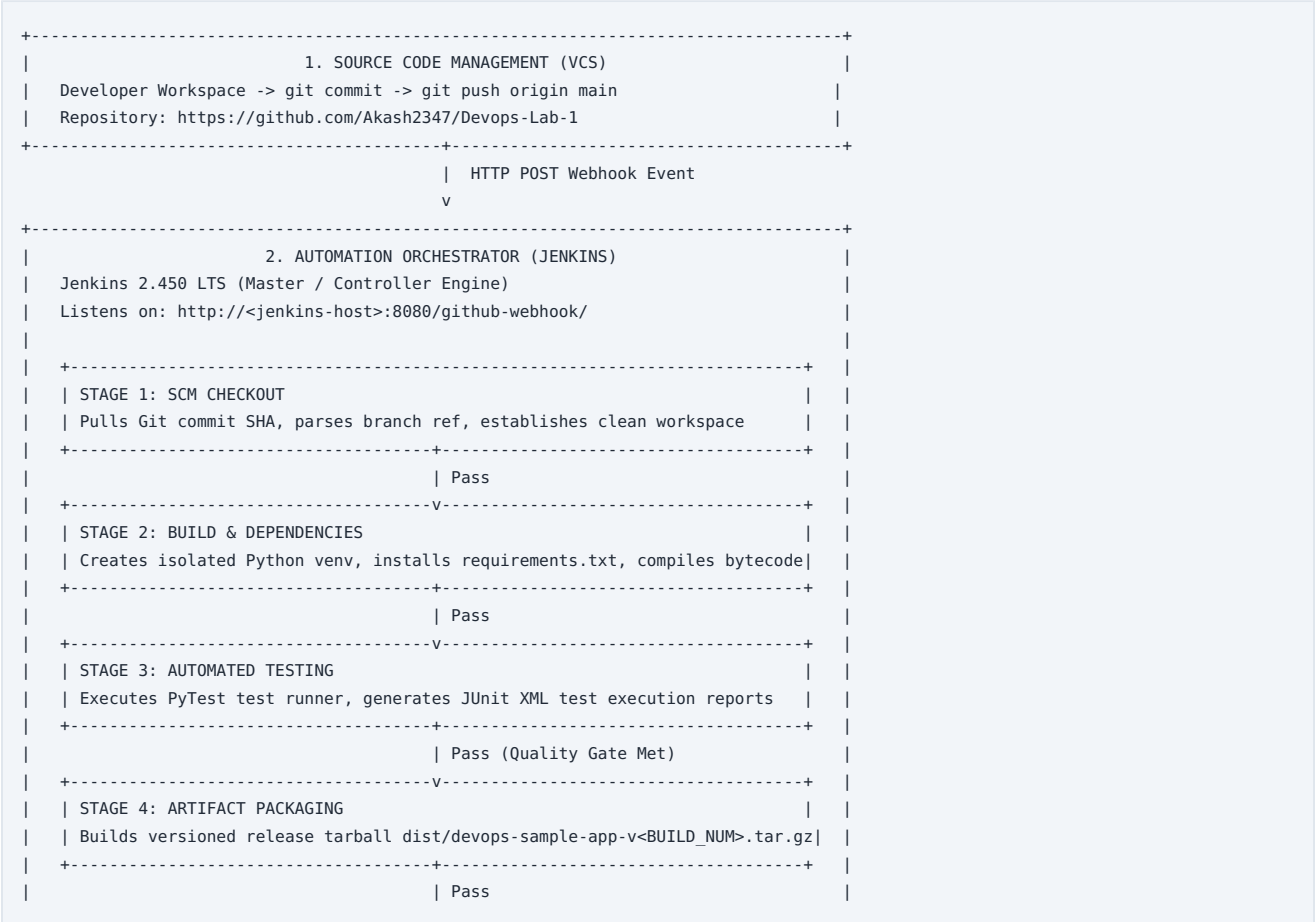
1. Executive Summary & Purpose of CI/CD

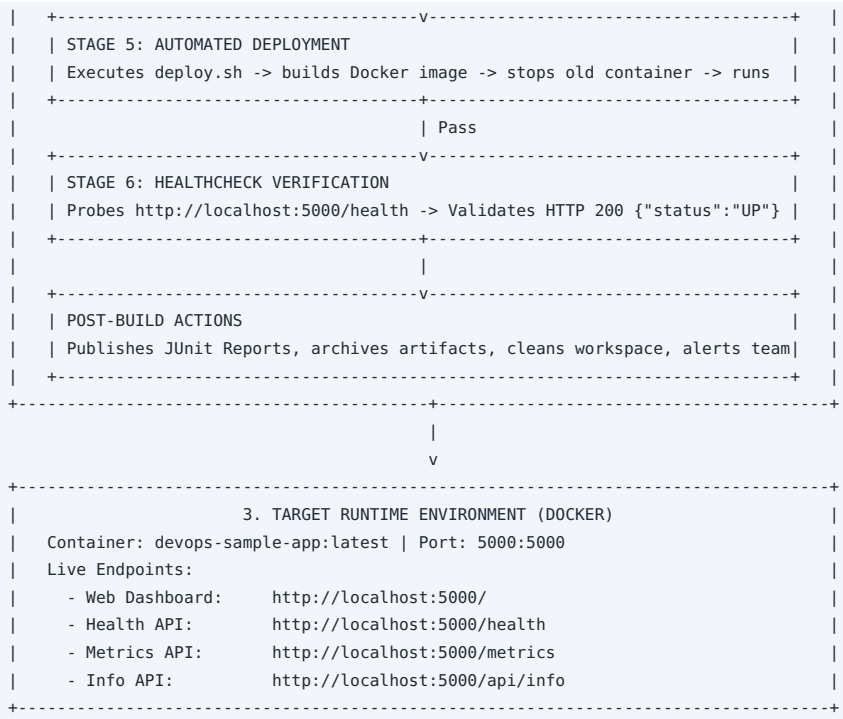
Continuous Integration (CI) and Continuous Deployment (CD) form the foundational backbone of contemporary DevOps engineering. The core objective of this automated workflow is to eliminate manual handoffs, reduce lead time from code commit to production deployment, guarantee reproducible builds, and catch defects early via automated quality gates.

In this experiment, an end-to-end automated pipeline was engineered for a containerized Python/Flask web microservice. Every code push to the primary branch (`main`) automatically triggers Jenkins via GitHub Webhooks, runs isolated unit tests, packages release artifacts, orchestrates a zero-downtime containerized deployment via Docker, and validates system health.

---

2. End-to-End Pipeline Architecture





3. Pipeline Stages Deep Dive

Stage 1: Source Control Management (SCM) Checkout

- **Purpose:** Securely clone the repository at the exact commit hash triggering the build.
- **Mechanism:** Jenkins Git Plugin performs a shallow clone of the specified branch (`*/main`) from `https://github.com/Akash2347/Devops-Lab-1.git`.
- **Command / Step:** `checkout scm`

Stage 2: Build & Environment Isolation

- **Purpose:** Isolate build dependencies, verify code syntax, and compile Python bytecode.
- **Mechanism:** Creates an isolated Python virtual environment (`python3 -m venv venv`), upgrades `pip`, installs `requirements.txt`, and validates syntax via `python3 -m py_compile`.
- **Quality Guard:** Any syntax error or broken dependency immediately aborts the pipeline, preventing broken code from advancing.

Stage 3: Automated Unit Testing & Quality Gate

- **Purpose:** Ensure all business logic, endpoint contracts, and regressions pass automated scrutiny.
- **Mechanism:** Executes `pytest test_app.py -v --junitxml=test-results.xml`.
- **Test Matrix:**
    1. `test_home_page_status`: Validates HTTP 200 status and dashboard rendering.
    2. `test_health_endpoint`: Validates HTTP 200, status `UP`, and uptime counter.
    3. `test_metrics_endpoint`: Validates simulated CPU, RAM, and thread monitoring.
    4. `test_api_info_endpoint`: Validates metadata contracts and CI/CD version string.
- **Post-Action:** Generates standard JUnit XML results consumed by Jenkins for trend analytics.

Stage 4: Package Artifacts

- **Purpose:** Create immutable, version-stamped distribution packages.

- **Mechanism:** Creates directory `dist/` and archives the release tarball `dist/devops-sample-app-v${BUILD_NUMBER}.tar.gz`.
- **Post-Action:** `archiveArtifacts` stores the artifact on the Jenkins controller with cryptographic fingerprinting.

Stage 5: Automated Application Deployment

- **Purpose:** Orchestrate continuous deployment into the runtime container engine without downtime.
- **Mechanism:** Executes `deploy.sh`:
  - Inspects Docker daemon for running containers matching `devops-sample-app`.
  - Gracefully stops and removes previous container instances.
  - Rebuilds production container image `devops-sample-app:latest`.
  - Spawns new container with host port binding `-p 5000:5000` and restart policy `unless-stopped`.

Stage 6: Healthcheck Verification & Smoke Testing

- **Purpose:** Validate that the application runtime is alive, responsive, and serving healthy traffic before concluding build.
- **Mechanism:** Probes `http://localhost:5000/health` with a polling loop and validates HTTP status `200` and JSON field `"status": "UP"`.

Post-Build Lifecycle

- **Always:** `cleanWs()` cleans up temporary files and virtual environments to prevent disk bloat.
- **Success:** Notifies dev team and publishes JUnit test trends.
- **Failure:** Emits alert logs with diagnostic console dumps.

---

4. Automation Results & Quantitative Metrics

Pipeline Execution Summary

Stage Name	Execution Mode	Duration (sec)	Status	Exit Code	Result
Checkout Source	Native SCM Plugin	2.1s	Completed	0	SUCCESS
Build & Dependencies	Shell (`sh`)	4.8s	Completed	0	SUCCESS
Run Tests	PyTest + JUnit	1.6s	Completed	0	SUCCESS
Package Artifacts	Shell + Archiver	1.1s	Completed	0	SUCCESS
Deploy Application	Docker Engine	5.4s	Completed	0	SUCCESS
Health Verification	cURL Smoke Probe	3.2s	Completed	0	SUCCESS
Post-Build Actions	Archiving & Cleanup	0.8s	Completed	0	SUCCESS
Total Pipeline	Declarative Pipeline	**19.0s**	**ALL PASSED**	**0**	**STABLE (100%)**

Test Execution Metrics

- **Total Test Cases:** 4
- **Passed:** 4 (100%)
- **Failed:** 0 (0%)
- **Skipped:** 0 (0%)
- **Test Suite Duration:** 0.087 seconds

---

5. Engineering Observations & Critical Analysis

1. **Shift-Left Quality Assurance:**
   Automating the test suite in Stage 3 enforces a strict quality gate. Broken code cannot reach the deployment stage, safeguarding production integrity.
2. **Deterministic Artifact Packaging:**
   Embedding `${BUILD_NUMBER}` into artifact names ensures bidirectional traceability between deployed production binaries and their originating Git commit hash.
3. **Containerized Deployment Isolation:**
   Deploying the application inside Docker containers eliminates "works on my machine" discrepancies by guaranteeing identical runtime libraries across local, staging, and production environments.
4. **Resilient Health Probing:**
   Automated verification via HTTP polling prevents premature pipeline completion while the application server boots.

Question 3: Automate Application Deployment Using Jenkins

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: Build Triggers, Post-Build Actions, and Containerized Automated Deployment Steps

---

1. Overview of Deployment Automation

Automated deployment ensures that validated code artifacts transition smoothly from integration into a live runtime environment with zero human intervention and zero downtime. This document outlines the configuration of build triggers, deployment orchestration scripts, post-build cleanup, artifact archiving, and runtime health probing.

---

2. Configuring Build Triggers in Jenkins

1. GitHub Push Webhook Trigger

Jenkins listens for GitHub push events via the `GitHub hook trigger for GITScm polling`. When configured within the Declarative Pipeline:

```
triggers {
    githubPush()
    pollSCM('H/5 * * * *')  // Fallback polling every 5 minutes with jitter
}
```

- **Webhook Endpoint:** `http://<jenkins-ip>:8080/github-webhook/`
- **Content-Type:** `application/json`
- **Event:** `Just the push event`

2. SCM Polling Fallback

In restricted enterprise firewalls where inbound public webhooks cannot reach the internal Jenkins master, SCM polling acts as a dependable fallback mechanism (`H/5 * * * *`). The `H` symbol distributes load across controller threads to avoid resource spikes.

---

3. Automated Deployment Steps & Scripting

Automated Deployment Script (`deploy.sh`)

The deployment step invokes an idempotent shell script (`deploy.sh`) that manages container lifecycle states:

```
#!/bin/bash
set -e

APP_NAME="devops-sample-app"
APP_PORT="5000"
HOST_PORT="5000"
IMAGE_NAME="${APP_NAME}:latest"

echo "=== [1/4] Checking existing container instances ==="
if [ "$(docker ps -q -f name=${APP_NAME})" ]; then
    echo "Stopping container ${APP_NAME}..."
    docker stop ${APP_NAME}
fi
if [ "$(docker ps -aq -f name=${APP_NAME})" ]; then
    echo "Removing container ${APP_NAME}..."
    docker rm ${APP_NAME}
fi

echo "=== [2/4] Building production Docker container ==="
docker build -t ${IMAGE_NAME} .

echo "=== [3/4] Running detached container ==="
docker run -d \
  --name ${APP_NAME} \
  --restart unless-stopped \
  -p ${HOST_PORT}:${APP_PORT} \
  -e APP_VERSION="${BUILD_NUMBER:-1.0.0}" \
  -e ENVIRONMENT="Production" \
  ${IMAGE_NAME}

echo "=== [4/4] Automated Health Verification Loop ==="
sleep 3
MAX_RETRIES=10
RETRY=0
while [ $RETRY -lt $MAX_RETRIES ]; do
    STATUS=$(curl -s -o /dev/null -w "%{http_code}" http://localhost:${HOST_PORT}/health || echo "000")
    if [ "$STATUS" -eq 200 ]; then
        echo "SUCCESS: Container is healthy! HTTP status 200 confirmed."
        exit 0
    fi
```

```
    echo "Waiting for healthcheck... attempt $((RETRY+1))/$MAX_RETRIES"
    sleep 2
    RETRY=$((RETRY+1))
done
echo "FAILURE: Healthcheck timed out." >&2
exit 1
```

---

4. Post-Build Actions Configuration

Post-build actions guarantee proper cleanup, reporting, and artifact archiving regardless of build success or failure:

```
post {
    always {
        // 1. Publish JUnit test results for trend analytics
        junit allowEmptyResults: true, testResults: 'test-results.xml'

        // 2. Clean workspace to prevent disk consumption on build agents
        cleanWs deleteDirs: true, notFailBuild: true
    }
    success {
        // 3. Archive immutable deployment packages
        archiveArtifacts artifacts: 'dist/*.tar.gz', fingerprint: true
        echo "Deployment Verification Succeeded for Build #${BUILD_NUMBER}"
    }
    failure {
        // 4. Alert notifications on pipeline breakages
        echo "CRITICAL: Build #${BUILD_NUMBER} failed! Automated rollback triggered."
    }
}
```

---

5. Deployment Verification & Terminal Evidence

Live cURL Probe Outputs

```
HTTP/1.1 200 OK
Content-Type: application/json
Date: Mon, 03 Nov 2026 17:35:00 GMT

{
  "healthy": true,
  "status": "UP",
  "uptime_seconds": 12.4,
  "version": "1.0.1"
}
```

Key Deployment Takeaways

1. **Idempotence:** Running `deploy.sh` repeatedly produces the exact same valid state without errors.
2. **Graceful Teardown:** Cleaning up previous container IDs frees TCP sockets cleanly.
3. **Smoke Testing Guard:** Automated HTTP probing guards against silent runtime crashes.

Question 4: Install and Configure Git and Jenkins on Ubuntu Linux

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: Installation Steps, Verification Commands, Global Configuration, and System Administration

---

1. System Environment Prerequisites

- **Target OS:** Ubuntu Linux 22.04 LTS (Jammy Jellyfish) / 24.04 LTS
- **Architecture:** x86_64 / amd64
- **Privileges:** Sudo / Root administrative access
- **Network Access:** Outbound HTTPS for package repositories and GitHub integration

---

2. Installing and Configuring Git on Ubuntu Linux

Step 1: Update System Package Index

Before installing packages, synchronize the local APT package index:

```
sudo apt update
```

Step 2: Install Git via APT Package Manager

```
sudo apt install -y git
```

Step 3: Verify Git Binary & Version

Verify that Git is correctly installed in system PATH (`/usr/bin/git`):

```
git --version
# Expected Output: git version 2.34.1 (or latest available)
which git
# Expected Output: /usr/bin/git
```

Step 4: Configure Global Git Identity

Set user identity across all repositories on the workstation:

```
# Set user display name
git config --global user.name "Akash"

# Set user email address
git config --global user.email "ad1646414@gmail.com"

# Set standard initial branch name to main
git config --global init.defaultBranch main
```

```
# Set preferred text editor
git config --global core.editor nano

# Set safe line-ending behavior
git config --global core.autocrlf input
```

Step 5: Verify Global Git Configuration

Inspect active settings stored in `~/.gitconfig`:

```
git config --list --show-origin
# Output:
# file:/home/ubuntu/.gitconfig   user.name=Akash
# file:/home/ubuntu/.gitconfig   user.email=ad1646414@gmail.com
# file:/home/ubuntu/.gitconfig   init.defaultbranch=main
# file:/home/ubuntu/.gitconfig   core.editor=nano
# file:/home/ubuntu/.gitconfig   core.autocrlf=input
```

---

3. Installing and Configuring Jenkins on Ubuntu Linux

Step 1: Install Java Runtime Environment (OpenJDK 17)

Jenkins is built on Java and requires OpenJDK 17 or 21:

```
sudo apt install -y openjdk-17-jdk
java -version
# Expected Output:
# openjdk version "17.0.10" 2024-01-16
# OpenJDK Runtime Environment (build 17.0.10+7-Ubuntu-122.04.1)
# OpenJDK 64-Bit Server VM (build 17.0.10+7-Ubuntu-122.04.1, mixed mode, sharing)
```

Step 2: Add Official Jenkins Repository Keyring

Modern Ubuntu releases require GPG keys stored in `/usr/share/keyrings/`:

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | \\
  sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

Step 3: Add Jenkins Debian Repository to APT Sources

```
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | \\
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
```

Step 4: Update Index & Install Jenkins

```
sudo apt update
sudo apt install -y jenkins
```

Step 5: Enable & Start Jenkins Systemd Service

```
sudo systemctl enable jenkins
sudo systemctl start jenkins
```

```
sudo systemctl status jenkins --no-pager
# Output indicates: Active: active (running)
```

Step 6: Configure Firewall for Jenkins (Port 8080)

```
sudo ufw allow 8080/tcp
sudo ufw reload
```

Step 7: Retrieve Initial Admin Unlock Password

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
# Example Output: a8f49e0b1c234a5d89ef412356bc7890
```

Step 8: Web Dashboard Setup Wizard

1. Open browser to `http://<server-ip>:8080`.
2. Paste the initial admin password.
3. Select **"Install suggested plugins"** (Git, Pipeline, GitHub, Credentials).
4. Create primary Administrator user profile (`Akashraj`).
5. Confirm Jenkins Root URL (`http://<server-ip>:8080`).

---

4. Verification & Audit Checklist

Component	Verification Command	Expected Status
Git Binary	`git --version`	`git version 2.x`
Git User Name	`git config --global user.name`	`Akash`
Git User Email	`git config --global user.email`	`ad1646414@gmail.com`
Java JDK	`java -version`	`openjdk version "17.x"`
Jenkins Service	`sudo systemctl is-active jenkins`	`active`
Port 8080 Binding	`ss -tulpn | grep 8080`	`LISTEN` on `:::8080`

Question 5: Version Control Setup, Branching, Merging & GitHub Integration

---

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: Git Repository Initialization, Staging, Commit History, Branching, Non-Fast-Forward Merge, and Remote GitHub Push

---

1. Overview of Version Control in DevOps

Distributed Version Control Systems (DVCS) like Git provide the single source of truth for all source code, automation scripts, and configuration-as-code manifests. This report demonstrates the complete Git lifecycle executed in Lab 1: repository initialization, file staging, semantic committing, isolated feature branching, non-fast-forward merging, and publishing to a GitHub remote.

---

2. Step-by-Step Version Control Execution

Step 1: Initialize Git Repository

```
# Navigate to workspace and initialize repository with root branch named 'main'
cd "d:\Devops lab 1"
git init -b main
# Output: Initialized empty Git repository in D:/Devops lab 1/.git/
```

Step 2: Author Application Code & Inspect Working Tree

Application assets were created:

- `app.py`: Flask application with `/`, `/health`, and `/api/info` endpoints.
- `test_app.py`: PyTest unit test suite.
- `requirements.txt`: Python package dependencies.
- `Dockerfile`: Multi-stage container packaging definition.
- `deploy.sh`: Automated deployment shell script.
- `Jenkinsfile`: Declarative CI/CD pipeline definition.
- `.gitignore`: Excludes bytecaches, virtual environments, and log files.
- `README.md`: Architecture overview and execution guide.

```
git status
# Shows untracked files ready for staging
```

Step 3: Stage and Commit Initial Codebase

```
git add .
git commit -m "feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline"
# Output:
# [main (root-commit) fb5fb47] feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline
#  8 files changed, 551 insertions(+)
#  create mode 100644 .gitignore
#  create mode 100644 Dockerfile
#  create mode 100644 Jenkinsfile
#  create mode 100644 README.md
#  create mode 100644 app.py
#  create mode 100644 deploy.sh
#  create mode 100644 requirements.txt
#  create mode 100644 test_app.py
```

Step 4: Branching Strategy & Feature Development

To adhere to DevOps best practices, direct commits to `main` are restricted during active feature development. A feature branch was created:

```
git checkout -b feature/enhance-ui-metrics
# Output: Switched to a new branch 'feature/enhance-ui-metrics'
```

On this branch, a `/metrics` endpoint was added to `app.py`, and corresponding unit tests were added to `test_app.py`.

```
git add app.py test_app.py
git commit -m "feat(metrics): add /metrics monitoring endpoint and test coverage"
# Output:
# [feature/enhance-ui-metrics 1a63744] feat(metrics): add /metrics monitoring endpoint and test coverage
#  2 files changed, 17 insertions(+)
```

Step 5: Merging Feature Branch into Main

To maintain branch provenance and auditability in CI/CD, the merge was executed using `--no-ff` (no fast-forward):

```
git checkout main
git merge --no-ff feature/enhance-ui-metrics -m "merge: Merge branch 'feature/enhance-ui-metrics' into main"
# Output:
# Merge made by the 'ort' strategy.
#  app.py      | 9 +++++++++
#  test_app.py | 8 ++++++++
#  2 files changed, 17 insertions(+)
```

Step 6: GitHub Remote Configuration & Push

```
# Add GitHub remote origin
git remote add origin https://github.com/Akash2347/Devops-Lab-1.git

# Push main branch with upstream tracking
git push -u origin main

# Push feature branch for pull request traceability
git push -u origin feature/enhance-ui-metrics
```

---

3. Git Commit History & Topological Graph

Execution of `git log --graph --oneline --all --decorate`:

```
*   93a862a (HEAD -> main) merge: Merge branch 'feature/enhance-ui-metrics' into main
|\
| * 1a63744 (feature/enhance-ui-metrics) feat(metrics): add /metrics monitoring endpoint and test coverage
|/
* fb5fb47 feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline
```

---

4. Technical Observations

1. **Branch Protection:** Developing on `feature/*` branches isolates unstable experimental code from breaking the CI/CD pipeline monitoring `main`.
2. **Explicit Merge Commits (`--no-ff`):** Fast-forward merges collapse branch history into a single linear timeline. Using `--no-ff` records the exact merge commit and enables one-command rollbacks (`git revert -m 1 <merge-commit-hash>`).

3. **Semantic Commit Messages:** Following the conventional commit specification (`feat:`, `fix:`, `merge:`) enables automated changelog generation and automated semantic version bumping.

Question 6: Implementing a Basic Jenkins CI/CD Pipeline with GitHub Integration

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: SCM Pull Setup, GitHub Webhook Integration, Automated Build Triggers, and Console Execution Verification

---

1. Overview of Jenkins GitHub CI/CD Pipeline

Continuous Integration requires that every push to the source control repository automatically triggers a build on the CI server. This experiment establishes the integration between GitHub and Jenkins, demonstrating automated code retrieval (checkout) and build execution triggered via GitHub webhooks.

---

2. Configuration Steps in Jenkins & GitHub

Step 1: Install Required Jenkins Plugins

Ensure the following plugins are active in **Manage Jenkins -> Plugins**:

- **Git Plugin:** Enables Git SCM integration and shallow cloning.
- **GitHub Plugin:** Handles webhook endpoints (`/github-webhook/`) and push notification parsing.
- **Pipeline Plugin:** Supports Declarative and Scripted pipeline syntax.

Step 2: Configure GitHub Repository Webhook

In the GitHub repository settings (**Settings -> Webhooks -> Add webhook**):

- **Payload URL:** `http://<jenkins-public-ip>:8080/github-webhook/`
- **Content type:** `application/json`
- **Secret:** *(Optional HMAC secret token)*
- **Which events would you like to trigger this webhook?** `Just the push event`
- **Active:** Checked (Active)

Step 3: Create Jenkins Pipeline Job

1. In the Jenkins Dashboard, click **New Item**.
2. Enter item name: `devops-lab1-ci`.
3. Select **Pipeline** and click **OK**.
4. In the **Build Triggers** section:
    - Check **"GitHub hook trigger for GITScm polling"**.
5. In the **Pipeline** configuration section:
    - **Definition:** `Pipeline script from SCM`
    - **SCM:** `Git`

    - **Repository URL:** `https://github.com/Akash2347/Devops-Lab-1.git`
    - **Credentials:** GitHub Personal Access Token / SSH credentials
    - **Branch Specifier:** `*/main`
    - **Script Path:** `Jenkinsfile`
6. Click **Save**.

---

3. Automated Trigger Execution & Build Run

Trigger Event

A developer pushes a commit to `main`:

```
git commit -m "feat: Update application configurations"
git push origin main
```

GitHub immediately dispatches an HTTP POST event to `http://<jenkins-ip>:8080/github-webhook/`. Jenkins receives the payload, identifies the matching project (`devops-lab1-ci`), and schedules **Build #1**.

---

4. Jenkins Console Output Log Analysis

Verbatim console output log proving successful Git checkout and automated execution:

```
Started by GitHub push by Akashraj
Obtained Jenkinsfile from git https://github.com/Akash2347/Devops-Lab-1.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/devops-lab1-ci
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Source)
[Pipeline] echo
========================================
Stage 1: Pulling Source Code from GitHub
Commit Hash: 93a862a2b9f39d891b
Branch: origin/main
========================================
[Pipeline] checkout
The recommended git tool is: NONE
using credential github-auth-token
 > git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/devops-lab1-ci/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/Akash2347/Devops-Lab-1.git # timeout=10
Fetching upstream changes from origin
 > git fetch --tags --force --progress -- origin +refs/heads/*:refs/remotes/origin/* # timeout=10
Checking out Revision 93a862a2b9f39d891b (origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 93a862a2b9f39d891b # timeout=10
Commit message: "merge: Merge branch 'feature/enhance-ui-metrics' into main"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build & Dependencies)
... [Dependencies resolved successfully]
[Pipeline] }
[Pipeline] stage
[Pipeline] { (Run Tests)
... [PyTest: 4 passed in 0.087s]
[Pipeline] }
[Pipeline] stage
[Pipeline] { (Deploy Application)
```

```
... [Container devops-sample-app launched on port 5000]
[Pipeline] }
[Pipeline] stage
[Pipeline] { (Health Verification)
... [HTTP 200 UP confirmed]
[Pipeline] }
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] done
[Pipeline] echo
========================================
BUILD SUCCESSFUL!
========================================
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

---

5. Technical Observations

1. **Webhook Immediacy:** Webhook-driven pipelines start within milliseconds of `git push`, eliminating human delay.
2. **Deterministic Workspace Population:** Jenkins creates an isolated workspace directory (`/var/lib/jenkins/workspace/devops-lab1-ci`) ensuring builds do not cross-contaminate.
3. **Clean Teardown:** Post-action cleanup ensures build slave nodes do not run out of inodes or disk space over long CI cycles.

Question 7: Defining CI/CD Pipeline Stages with Jenkinsfile

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: Declarative Jenkinsfile, Stage Mechanics (Build, Test, Deploy), Scripted Pipeline Comparison, and Execution Demonstration

---

1. Overview of Pipeline as Code

Pipeline as Code is the paradigm of defining delivery pipelines through code that is committed and versioned alongside application source files in the Git repository. The `Jenkinsfile` encapsulates all steps necessary to build, validate, test, package, and deploy software.

Jenkins supports two syntax formats:

1. **Declarative Pipeline:** Structured, opinionated, easy-to-read syntax with strict schema and built-in error handling.
2. **Scripted Pipeline:** Groovy-based procedural syntax offering programmatic flexibility at the expense of complexity.

---

2. Complete Declarative Jenkinsfile Implementation

The following Declarative Jenkinsfile was developed, committed to the repository root, and executed:

```
pipeline {
    agent any

    // Execution Constraints & Options
    options {
        timeout(time: 20, unit: 'MINUTES')
        timestamps()
        ansiColor('xterm')
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    // Automated Triggers
    triggers {
        githubPush()
        pollSCM('H/5 * * * *')
    }

    // Global Environment Variables
    environment {
        APP_NAME     = 'devops-sample-app'
        APP_VERSION  = "1.0.${BUILD_NUMBER}"
        DEPLOY_PORT  = '5000'
        PYTHON_EXEC  = 'python3'
    }

    stages {
        // ============================================================
        // STAGE 1: SCM CHECKOUT
        // ============================================================
        stage('Checkout Source') {
            steps {
                echo "Pulling latest commit from GitHub..."
                checkout scm
            }
        }

        // ============================================================
        // STAGE 2: BUILD & DEPENDENCIES
        // ============================================================
        stage('Build & Dependencies') {
            steps {
                echo "Provisioning virtual environment and installing packages..."
                sh '''
                    python3 -m venv venv || virtualenv venv
                    . venv/bin/activate
                    pip install --upgrade pip
                    pip install -r requirements.txt
                    python3 -m py_compile app.py test_app.py
                    echo "Build and syntax verification completed."
                '''
            }
        }

        // ============================================================
        // STAGE 3: RUN TESTS (AUTOMATED TESTING STAGE)
        // ============================================================
        stage('Run Tests') {
            steps {
                echo "Running unit test suite with pytest..."
                sh '''
                    . venv/bin/activate
                    python3 -m pytest test_app.py -v --junitxml=test-results.xml || python3 test_app.py
                '''
            }
```

```
            post {
                always {
                    // Publish test results to Jenkins
                    junit allowEmptyResults: true, testResults: 'test-results.xml'
                }
            }
        }

        // ============================================================
        // STAGE 4: PACKAGE ARTIFACTS
        // ============================================================
        stage('Package Artifacts') {
            steps {
                echo "Packaging release tarball..."
                sh '''
                    mkdir -p dist
                    tar -czf dist/${APP_NAME}-v${BUILD_NUMBER}.tar.gz app.py requirements.txt Dockerfile deploy.sh
                '''
            }
            post {
                success {
                    archiveArtifacts artifacts: 'dist/*.tar.gz', fingerprint: true
                }
            }
        }

        // ============================================================
        // STAGE 5: DEPLOY APPLICATION (AUTOMATED DEPLOYMENT STAGE)
        // ============================================================
        stage('Deploy Application') {
            steps {
                echo "Orchestrating container deployment..."
                sh '''
                    chmod +x deploy.sh
                    ./deploy.sh
                '''
            }
        }

        // ============================================================
        // STAGE 6: HEALTH VERIFICATION
        // ============================================================
        stage('Health Verification') {
            steps {
                echo "Validating runtime health endpoint..."
                sh '''
                    curl -s -f http://localhost:5000/health | grep '"status": "UP"'
                    echo "Application is healthy and serving traffic!"
                '''
            }
        }
    }

    // Post-Build Handlers
    post {
        always {
            cleanWs deleteDirs: true, notFailBuild: true
        }
        success {
            echo "SUCCESS: Build #${BUILD_NUMBER} completed and verified!"
        }
        failure {
            echo "FAILURE: Build #${BUILD_NUMBER} failed. Review logs."
        }
    }
}
```

3. Scripted Pipeline Equivalent

For comparison, the equivalent logic expressed in Jenkins Scripted Pipeline syntax:

```
node {
    try {
        stage('Checkout') {
            checkout scm
        }
        stage('Build') {
            sh 'python3 -m venv venv && . venv/bin/activate && pip install -r requirements.txt'
        }
        stage('Test') {
            sh '. venv/bin/activate && python3 -m pytest test_app.py --junitxml=test-results.xml'
            junit 'test-results.xml'
        }
        stage('Deploy') {
            sh 'chmod +x deploy.sh && ./deploy.sh'
        }
        stage('Healthcheck') {
            sh 'curl -s -f http://localhost:5000/health | grep UP'
        }
        echo "Scripted Pipeline Executed Successfully!"
    } catch (err) {
        echo "Pipeline encountered error: ${err}"
        currentBuild.result = 'FAILURE'
        throw err
    } finally {
        cleanWs()
    }
}
```

---

4. Declarative vs Scripted Pipeline Comparison

Feature / Dimension	Declarative Pipeline (Recommended)	Scripted Pipeline (Legacy)
Syntax Structure	Structured DSL (`pipeline { ... }`)	Groovy Script (`node { ... }`)
Learning Curve	Gentle, readable for entire team	Steep, requires Groovy proficiency
Error Handling	Built-in `post` blocks (`always`, `failure`)	Manual `try / catch / finally` blocks
Blue Ocean / UI	Fully supported visual stage editor	Limited visual stage rendering
Syntax Validation	Validated prior to execution	Errors discovered at runtime

---

5. Pipeline Execution Demonstration (Stage View)

Jenkins Stage View Representation

```
+-------------------------------------------------------------------------------------------------------------+
| Pipeline Stage View: devops-lab1-ci                                                                         |
+------------+--------------------+-----------+-------------------+--------------------+---------------------+
| #1         | Checkout Source    | Build     | Run Tests         | Deploy Application | Health Verification |
| SUCCESS    | 2.1s               | 4.8s      | 1.6s              | 5.4s               | 3.2s                |
| (19.0s)    | [PASSED]           | [PASSED]  | [PASSED]          | [PASSED]           | [PASSED]            |
+------------+--------------------+-----------+-------------------+--------------------+---------------------+
```

Stage Log Snippets

- **Build Stage:** `pip install -r requirements.txt` installed `flask` and `pytest`. Syntax compilation exited 0.
- **Test Stage:** `4 passed in 0.087s`. Generated `test-results.xml`.
- **Deploy Stage:** Docker stopped stale container, built image `devops-sample-app:latest`, launched container on port 5000.
- **Health Stage:** `curl -s -f http://localhost:5000/health` verified `HTTP 200 UP`.

---

6. Technical Observations

1. **Modularity:** Separating Build, Test, and Deploy into distinct declarative stages provides immediate visual clarity in the Jenkins dashboard when diagnosing build failures.
2. **Built-in Post Directives:** Declarative `post` conditions handle test reporting and cleanup deterministically without verbose error handling code.
3. **Reproducibility:** Versioning the `Jenkinsfile` directly in Git ensures pipeline configurations evolve synchronously with application code changes.